Foundation · for every age group

Introduction to Actuators





Sensors notice. Actuators *do*.

Servo SG90 | Ultrasonic HC-SR04 | map() maths

Did you know?

Your robot can have the best sensors in the world. Without an actuator it can still only sit there and know things.

Let's think...

What actually moves?

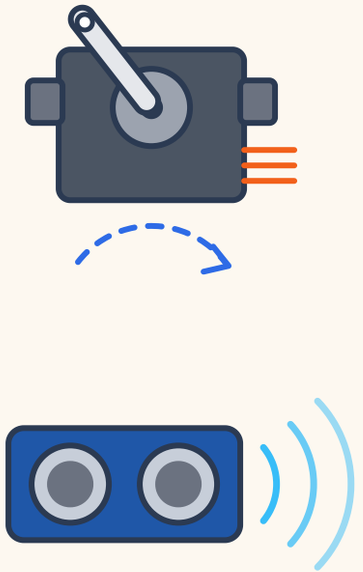
A robot that only watches

You build a robot with a camera, a distance sensor and a thermometer. It can tell you the door is open, someone is 30 cm away and the room is 31 °C. Then it does **nothing at all**, because nothing on it can move.

Shout at it?

Push it by hand?

Give it muscles!



Today: we add the muscles, and we make them obey a sensor.

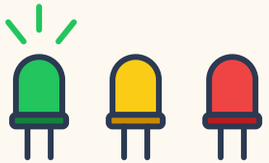
Learning aims

Today I will learn to…

1. Say what an **actuator** is and name three kinds
2. Explain how a **servo** knows its own angle
3. Read a distance with an **ultrasonic** sensor
4. Use **map()** to turn centimetres into degrees
5. Show the angle on **three LEDs**

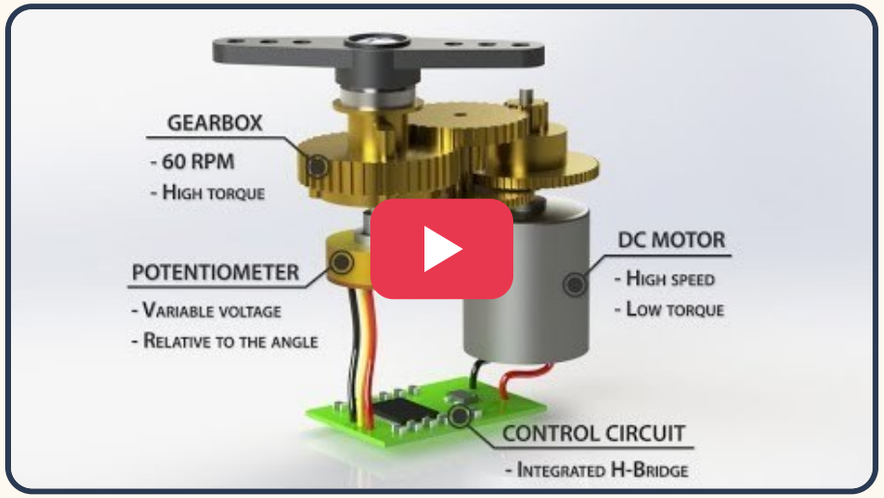
Keywords

actuator · servo · PWM · feedback · map() · constrain · degree



Watch first

Inside a servo motor



How Servo Motors Work

How To Mechatronics · YouTube

Watch how the little motor, the gears and the hidden potentiometer work together, then how the Arduino tells it an angle.

youtu.be/LXURLvga8bQ

While you watch

1. What is inside the servo besides a motor?
2. What does the servo do if you push its arm?
3. How many wires does it need?

On the portal

The same video plays inside the Lab Planner portal, on this lesson's page.

The big idea

Sensor in, actuator out

SENSE ultrasonic → **THINK** Arduino code → **ACT** servo + LEDs

The rule

A **sensor** turns something in the world into **electricity the Arduino can read**.
An **actuator** turns **electricity from the Arduino** back into something that happens in the world: movement, light, sound, heat.

Energy goes in, work comes out. That is why actuators need more current than sensors.

Sensors you know

LDR · light
HC-SR04 · distance
DHT22 · temperature
Push button · a finger

Actuators you know

LED · light
Buzzer · sound
DC motor · spin
Servo · **an exact angle**

Careful

An LED is an actuator too. It is the smallest one in your kit.

Current

Arduino pin: 40 mA maximum. A motor wants ten times that, so it gets its own supply.

The family

Six kinds of actuator



DC motor

Spins and spins

Wheels, fans



Servo

Holds an exact angle

Arms, steering



Stepper

Moves in exact steps

3D printers

Spot the odd one

Every one of these turns electricity into movement. Only the servo is told an angle and stays there.



Solenoid

Pushes or pulls once

Door locks



Relay

Switches big things on

Lamps, pumps



Pump / valve

Moves water or air

Irrigation

Also actuators

Speaker · sound

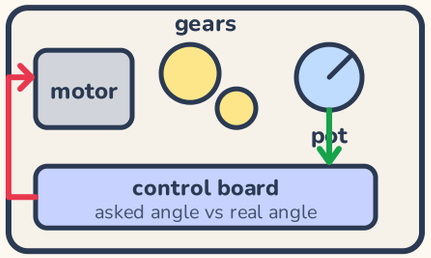
Heater · heat

Screen · light

Electromagnet · force

The science

What is inside a servo?



it checks itself, 50 times a second

Four parts in one box

1. A small DC motor, far too fast and too weak on its own
2. A **gear train** that trades speed for turning force
3. A **potentiometer** on the output shaft
4. A control board that reads your signal

Feedback: the clever bit

The potentiometer tells the board **where the arm really is**. The board compares that with **the angle you asked for**. If they differ it drives the motor until they match, then stops.

asked 90°, reads 70° → keep turning

Try this with a real servo

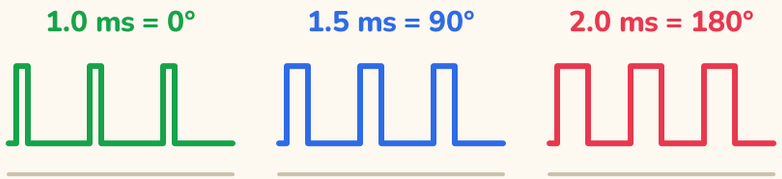
Power it, send `write(90)`, then gently try to turn the arm with your finger. You can feel it push back. That is feedback you can touch.

So a servo cannot…

… spin round and round. A standard SG90 turns about **180°** and no further.
Forcing it past the stop strips the plastic gears. If you need endless spinning, you want a DC motor or a **continuous-rotation** servo, which is a different part.

The signal

How you tell a servo an angle



one pulse every 20 ms (50 times a second)

One wire, one message

The orange wire carries a pulse **50 times a second**. The servo does not measure the voltage, it measures **how long each pulse stays high**.

Good news: `Servo.h` does all of this for you. You write `arm.write(135)` and the library makes the right pulse.

The three wires

Wire	Goes to
Brown	GND
Red	5V
Orange	any digital pin

In code

```
#include <Servo.h>
Servo arm;

arm.attach(6);
arm.write(0);   // one end
arm.write(90);  // middle
arm.write(180); // other end
```

Speed

A servo takes about 0.1 s to swing 60°. Ask it to move faster than that and it simply cannot keep up.

Power warning

One small servo moving slowly is fine on the Arduino 5V pin.

Two or more servos, or anything carrying a load, will pull the 5V rail down and reset your board. Give them a separate battery pack and **join the grounds**.

Real world

Actuators you have already met

Car windows

A small motor and a worm gear lift the glass and hold it

3D printers

Steppers move the head to an exact spot, thousands of times

Factory arms

Servos at every joint, each one holding its own angle

Drone gimbals

Servos tilt the camera the opposite way to the wind

? Find three actuators in this room before the end of the lesson. A fan, a door closer, a printer, a phone that buzzes: all of them count.

Same pattern

Sense, decide, act. Every machine in the list does the same three things your project will do today.

In Pakistan: the automatic doors at a shopping mall, the barrier at a car park, and the motorised gate on a street are all sensor plus actuator. Watch one on the way home.

Choosing

Motor, servo or stepper?

	DC motor	Servo	Stepper
How you control it	On, off, speed	**An angle, 0-180°**	A number of steps
Does it know where it is?	No	**Yes, it has feedback**	It counts, if nothing slips
Best for	Wheels and fans	**Arms, steering, doors**	Printers and plotters
Extra parts needed	Motor driver	**None, just a pin**	Driver board
Typical price	Low	**Low**	Higher

For today

Your project needs an exact angle that follows a distance. That is a servo's whole job.



Choose a servo when the question is "which way should it point?" Choose a motor when the question is "how fast should it go?"

Check yourself

Four quick questions

1 An actuator turns _____ into _____.

electricity into movement, light, sound or heat

2 Which part tells the servo where its arm really is?

the potentiometer on the output shaft

3 How often does the control pulse repeat?

50 times a second, every 20 ms

4 Why must two servos not share the Arduino's 5V pin?

they pull more current than it can give, and the board resets

Teacher

Cover the handwriting with your hand, ask the class, then reveal.

Quick scores

4 right · you are ready to build

2-3 right · look again at the inside of the servo

0-1 right · watch the video once more

Part 2

Make it

A servo that follows your hand, and three LEDs that say where it is pointing.

45 minutes | Pairs | Simulator or real kit



The build

Kit list and wiring

Part	Pin on the part	Arduino
HC-SR04	VCC	5V
	Trig	**9**
	Echo	**10**
	GND	GND
Servo SG90	Red	5V
	Brown	GND
	Orange	**6**
Green LED	+ leg, 220 Ω	**2**
Yellow LED	+ leg, 220 Ω	**3**
Red LED	+ leg, 220 Ω	**4**

What you need

Arduino Uno + USB cable · breadboard · HC-SR04 · servo SG90 · 3 LEDs (green, yellow, red) · 3 × 220 Ω resistors · jumper wires

No kit? Everything below works in Circuit Lab in the browser.

Open the ready-made circuit

In the portal, open **F-04** and press **Try it in a simulator**.

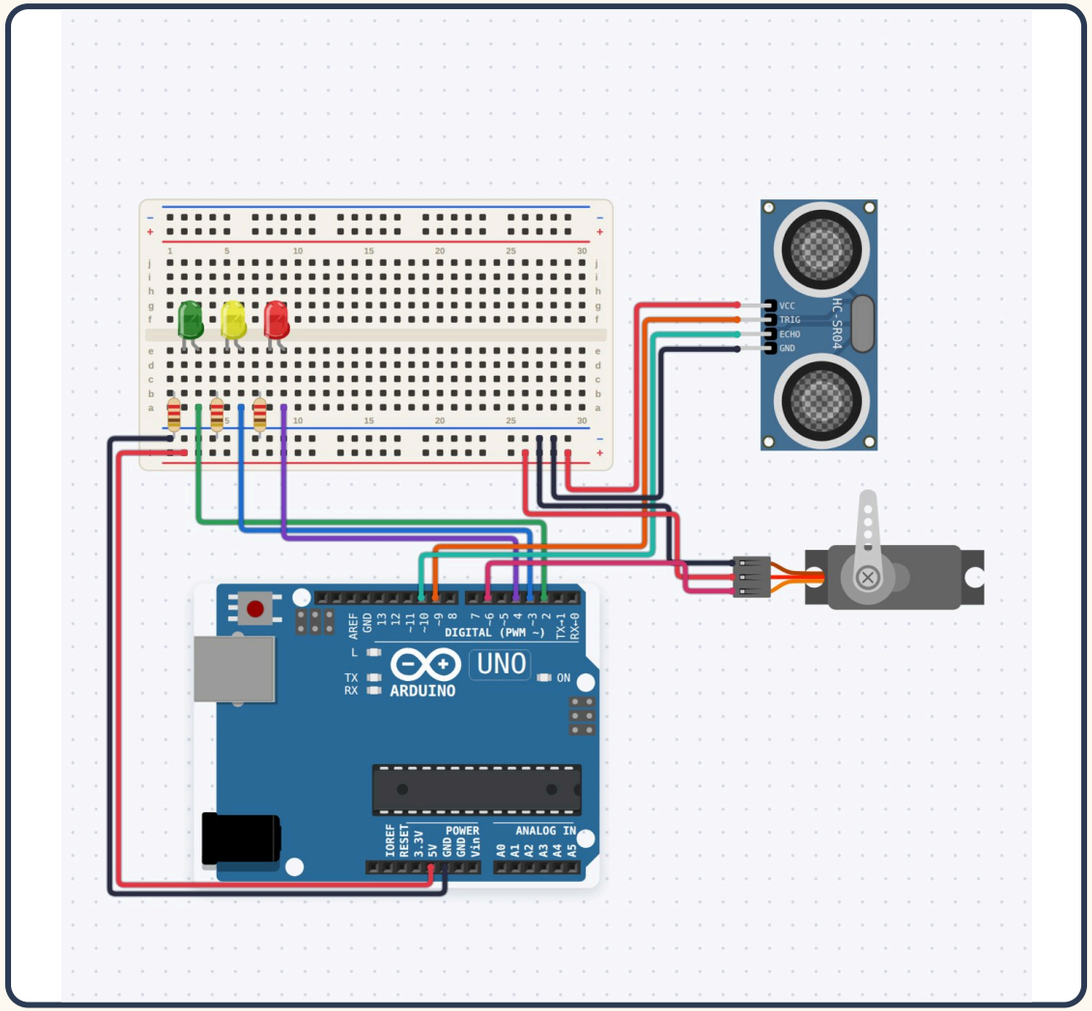
`is.gd/circuitlab` → the card marked **F-04**

Do not skip

Every LED needs its own resistor. Three LEDs sharing one resistor go dim when two are on at once.

The build

The finished circuit



Build it in this order

1. Power rails first: 5V to the red rail, GND to the blue rail.
2. Three LEDs on the board, long leg to a resistor, resistor to pins 2, 3 and 4.
3. Short legs to the blue rail.
4. HC-SR04: VCC and GND to the rails, Trig to 9, Echo to 10.
5. Servo last: brown to GND, red to 5V, orange to pin 6.

Check before you power up

Count the ground wires. The sensor, the servo and all three LEDs must reach the **same** GND. A missing ground is the fault you will meet most often this year.

Simulator tip

In the simulator, drag the little ball in front of the sensor to change the distance.

The code

Distance in, angle out

```ino
#include <Servo.h>
const int TRIG = 9, ECHO = 10, SERVO = 6;
const int GREEN = 2, YELLOW = 3, RED = 4;
const int NEAR_CM = 10, FAR_CM = 60;

Servo arm;

long readDistanceCm() {
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);
  if (us == 0) return FAR_CM;   // nothing came back
  return us / 58;               // there and back
}
```

```ino
void loop() {
  long cm = readDistanceCm();
  cm = constrain(cm, NEAR_CM, FAR_CM);

  int angle = map(cm, FAR_CM, NEAR_CM, 0, 180);
  arm.write(angle);

  digitalWrite(GREEN,  angle < 60);
  digitalWrite(YELLOW, angle >= 60 && angle < 120);
  digitalWrite(RED,    angle >= 120);

  Serial.println(angle);
  delay(100);                   // let it move
}
```

Read it top to bottom. Nothing here is magic.

pulseIn measures how long the echo takes, in millionths of a second. Sound covers 1 cm there and back in about 58 of them.

map(cm, 60, 10, 0, 180) reads as: when cm is 60 give 0, when cm is 10 give 180, and slide smoothly in between.

The maths

Turning centimetres into degrees

The one line that matters

```
angle = map(cm, 60, 10, 0, 180);
```

Two ranges, lined up: **60 → 0** and **10 → 180**. Everything between follows the straight line.

Why constrain() comes first

A person standing 3 m away gives cm = 300. Without `constrain`, map would return **–864°** and the servo would slam to its end stop and buzz. constrain clips the reading to the window first.

Work one out by hand

Say the object is at **35 cm**.

How far into the window is it? 60 – 35 = **25 cm**
How wide is the window? 60 – 10 = **50 cm**
So we are **25 ÷ 50 = half** way
Half of 180° = **90°**

map() just did that arithmetic for you.

Change the feel of it

You want	Change
React only up close	FAR_CM = 30
Turn the other way	map(cm, 10, 60, 0, 180)
Use only half the sweep	0, 180 → 45, 135
Calmer movement	delay(100) → delay(200)

Test it

Does it do what we predicted?

Hand at this distance	Servo should read	LED on	Tick
60 cm or more	**0°**	**Green**	☐
50 cm	**36°**	**Green**	☐
40 cm	**72°**	**Yellow**	☐
30 cm	**108°**	**Yellow**	☐
20 cm	**144°**	**Red**	☐
10 cm or less	**180°**	**Red**	☐

Open the Serial Monitor while you test. It prints the distance and the angle on every reading, so you can see what the board actually thinks.

If it misbehaves

Servo twitches all the time → the reading jumps a little; add a deadband (challenge 4)
Always 180° → Echo and Trig swapped, or nothing in range
Board keeps resetting → the servo is pulling too much current
LEDs all off → check the ground rail

Evidence to keep

Tick every row above, then write one sentence: *what happens between 40 and 30 cm, and why?*

Push further

Five challenges

1 **Flip it** Make the servo turn the other way as you get closer. One line changes.

2 **Add a beep** Add a buzzer on pin 8 that beeps faster the closer you get.

3 **Smooth it** Move 1° at a time towards the target instead of jumping straight there.

4 **Stop the twitch** Only move if the new angle differs from the old one by more than 3°.

5 **Scanner** Sweep the servo 0 → 180 on its own and print the distance at every angle. You have built a radar.

Where this goes

Challenge 5 is how a real parking sensor and a lidar both work: point, measure, turn a little, measure again.

Finished early?

Swap the LEDs for a buzzer and a relay, and you have the start of an automatic door: something approaches, the door opens, and it closes again when the way is clear.

Wrap up

What you can do now

- ✓ Say what an actuator is and name three kinds
- ✓ Explain how a servo knows its own angle
- ✓ Read a distance with an ultrasonic sensor
- ✓ Use map() to turn centimetres into degrees
- ✓ Show the angle on three LEDs

Before you go

Keep your circuit saved in Circuit Lab. Next lesson starts from it.

In one sentence

A sensor measured the world, map() turned that number into an angle, and the actuator made it real.

Next lesson

We drive something heavier than a servo: a DC motor through a driver board, where the Arduino cannot supply the current and has to ask a bigger circuit to do the work.